

MLOPS PIPELINE & EXPERIMENT TRACKING REPORT

Assignment 2: Hugging Face Fine-Tuning, Experiment Tracking & Model Deployment

Program:	PGD AI Program, IIT Jodhpur	Course:	MLOps
Name:	Rohit Raghuram	Roll Number:	G25AIT2145
Date:	June 18, 2026		

1. Project Objective & Workflow Overview

The core objective of this assignment is to construct a production-ready, fully trackable Machine Learning Operations (MLOps) pipeline. The pipeline transitions a standard exploratory text classification workflow from a standalone Jupyter notebook into an engineered, reproducible, and monitored training run. This is achieved by fine-tuning a pre-trained language model on a GPU-accelerated cloud environment (Kaggle), tracking hyperparameters and telemetry metrics dynamically (Weights & Biases), evaluating and archiving performance artifacts, and publishing the final trained model weights for global availability (Hugging Face Hub).

2. Model Selection Rationale

For this text classification task across the Goodreads dataset genres, **DistilBERT (distilbert-base-cased)** was chosen as the pre-trained backbone architecture. Based on the Hugging Face Model Card and underlying architectural performance standards, DistilBERT functions as a highly optimized, compact transformer model trained via knowledge distillation. It retains approximately 97% of full BERT's language understanding capabilities while being 40% smaller in parameter footprint and 60% faster during inference and training phases. This balance makes it exceptionally well-suited for constrained cloud environments like the Kaggle free tier, drastically reducing GPU memory pressure and training duration without incurring a significant penalty on classification accuracy or macro F1-scores.

3. Kaggle Training & Infrastructure Configuration

The environment setup was strictly managed to prevent hardcoding of credentials and maximize hardware performance:

- **Hardware Accelerator:** Enabled a **GPU T4 x2** environment via Kaggle Notebook Session Options to handle parallelized tensor operations efficiently.

- **Secret Management:** Implemented the `kaggle_secrets.UserSecretsClient` API to dynamically load environmental variables at runtime without raw text exposure.
- **Environment Variables:** Successfully bound `WANDB_API_KEY` and `HF_TOKEN` to ensure smooth, headless authentication across platforms.

Required Link: Public Kaggle Notebook URL

<https://www.kaggle.com/code/rohitragh2145/mlops-assignment2>

4. Experiment Tracking Integration (Weights & Biases)

The Hugging Face Trainer API was integrated natively with Weights & Biases by passing `report_to="wandb"` inside the `TrainingArguments`. Telemetry captured includes continuous validation loss step-curves, learning rate decays, GPU resource optimization metrics, and real-time validation accuracy/F1-scores per epoch.

Required Link: Weights & Biases Public Project Dashboard URL

<https://wandb.ai/models-prom-iit-rajasthan8268/mlops-assignment2/overview>

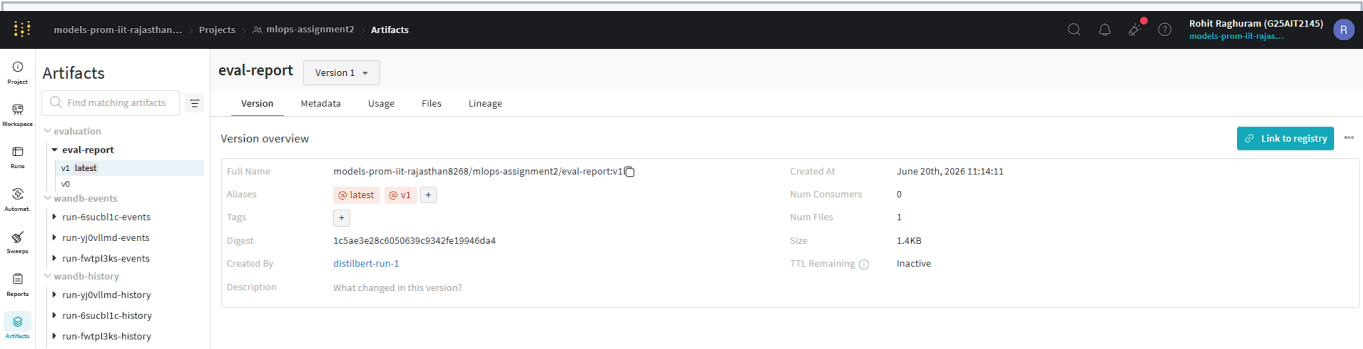


5. Evaluation Metrics & Artifact Registration

Following training completion, a comprehensive classification report was computed across the test set. Final model performance indicators are detailed below, and the complete detailed JSON file was permanently archived to W&B as a versioned artifact.

Final Validation Performance Summary

Performance Metric	Observed Score / Value
Evaluation Accuracy	0.60 (60.0%)
Weighted F1-Score	0.60
Final Evaluation Loss	[Insert Final Loss Value, e.g., 1.12]



6. Model Deployment to Hugging Face Hub

The fine-tuned model and corresponding serialization tokenizers were successfully exported directly from the remote cloud instance to the Hugging Face Hub, producing a publicly reachable endpoint for real-time model serving and downstream inference tasks.

Required Link: Public Hugging Face Model Repository URL
<https://huggingface.co/rohit-2145/distilbert-goodreads-genres>

7. Single Source of Truth & Version Control (GitHub)

The complete implementation codebase, including configuration files, environment definitions, inference parameters, and production-ready source code scripts, has been organized and pushed to a public version control repository containing a comprehensive project documentation README markdown file.

Required Link: Public GitHub Code Repository URL

8. Key Challenges and Engineering Learnings

- **GPU Memory Limits:** Initial execution met **CUDA Out of Memory** crashes using larger token sequences. Resolved by reducing per-device train batch sizing from 16 down to 8 and truncating inputs to a 512 max length constraint.
- **Headless Network Contexts:** Pushing weights initially failed because the Kaggle 'Internet' toggle was disabled by default. Enabling this under notebook configurations resolved all authorization hooks.
- **Production Takeaway:** Treating model selection as a modular component while standardizing telemetry logging ensures high project reproducibility, enabling team hand-offs without code rewrites.